Final Project

StockBear / Chess

June 30, 2025

1 Statement of the Problem

The game of chess, while rooted in centuries of tradition, poses significant challenges in its transference to a program. The creation of a computerized chess game requires careful consideration and effective implementation of game theory, complex algorithms, strategies, and in my version, artificial intelligence (AI). .

The primary problem consists of the user interface. The program should be intuitive, engaging and visually appealing to maintain the player's interest while appropriately showcasing the sophisticated strategies of the underlying chess engine.

Another crucial aspect is developing an AI chess engine that can not merely participate in the game, but can also pose substantial challenges to an opponent, replicating the strategic thinking and unpredictability of a human player. Building an AI engine that can choose from dozens of possible moves, anticipate various strategies, and react to the player's actions while remaining confined to the rules of chess is a formidable task.

Overall, the problem is to design and execute a strong, strategic chess engine and present it in an engaging, user-friendly game environment. The goal is to create an intelligent and challenging opponent within an appealing digital game of chess, ensuring an engaging experience for players of varying skill levels.

2 Description of Solution

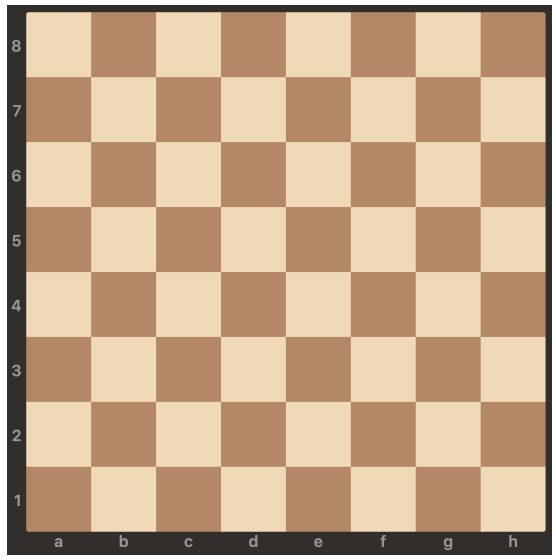
2.1 Creating Chess

PREAMBLE: some values in this program are constants that were either chosen arbitrarily or experimentally that control various aesthetic aspects of my game. They are as follows:

```
ofColor BLACK(0, 0, 0);
ofColor WHITE(255, 255, 255);
ofColor Default[2] = { WHITE,BLACK };
ofColor LITEBROWN(196, 170, 137);
ofColor DARKBROWN(97, 68, 48);
ofColor WoodenTheme[2] = { LITEBROWN, DARKBROWN};
```

```
ofColor CHECKRED(174, 13, 45);
ofColor KINGCHECK(235, 30, 15);
const int n = 8; // chess
int boardWidth = WINLEN - 2*XDIS;
int squareLen = boardWidth / 8;
int YDIS = 25;// WINHYT / 9;
int XDIS = 213;// WINLEN / 9;
```

The first step to creating chess is drawing the board:



This shows the array of ofRectangles i used to visually represent the chess board in my program, which was initialized in the of setup() function:

```
ofRectangle BORDRECTS[64];
//in setup function:
for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n; j++)
   {
      BORDRECTS[n * (i)+j].set(j * squareLen + (XDIS), i * squareLen + (YDIS /
         2), squareLen, squareLen);
      //BORDRECTS[n * (i)+j] = ofRectangle(j * squareLen + (XDIS), i * squareLen
         + (YDIS / 2), squareLen, squareLen);
   }
}
```

This made the board be drawn in the center of the window, and gave me a specific object to manipulate what would be shown on screen around. The array's indices correspond to

specific squares, with 0 representing the a8 square (upper left), and 63 representing h1 (bottom right).

2.2 Interacting with the Board

The next step I implemented is a nifty selection feature. In openframeworks there is a member function to the ofApp class called mousePressed that is called whenever the mouse is clicked and gives tell you wher eit was clicked. I have two global variables called selectIndex and destIndex that determine things move info and handle selecting things on the board. It does this by iterating over every rectangle in the board everytime you click, and checking if you clicked in any of them.

```
void select(int x, int y)
{
   for (int i = 0; i < n * n; i++)
   {
      if (BORDRECTS[i].inside(x, y))
      {
         if (destIndex == i) destIndex = -1; //unselects square if you click on
             the same one
         else
         {
            selectIndex = destIndex;
            destIndex = i;
         }
      }
   }
}
```

This makes it so that whenever you click on the board, that square will be selected, and the last square you clicked is still remembered. It is visualized in the draw function by changing the color to CHECKRED (see preamble) whenever the square that has the same index as destIndex is being drawn.

```
void select(int x, int y)
{
   for (int i = 0; i < n * n; i++)
   {
      if (BORDRECTS[i].inside(x, y))
      {
         if (destIndex == i) destIndex = -1;
         else
         {
            selectIndex = destIndex;
```

```
            destIndex = i;
          }
        }
    }
}
```

2.3 Representing the Board

With this we have a chess board that we can click about! One more thing to implement is the actual pieces. To represent the board, we will use a matrix of integers that represent the different pieces. The architecture I went with defines the team as a 8 or a 16, and the piece as 0 for nothing, 1 for pawn, 2 for knight and so on. So the starting position would look something like this in memory:

12 10 11 13 14 11 10 12

9 9 9 9 9 9 9 9

0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0

0 0 0 0 0 0 0 0

17 17 17 17 17 17 17 17

20 18 19 21 22 19 18 20

2.4 Drawing the Board

One final step until we have something that looks like chess is to draw the board, but draw the pieces where they are in our representation. To do this, I ripped piece pngs from chess.com, and created the relevant objects:

```
ofImage whitePawn;
ofImage blackPawn;
ofImage whiteKnight;
ofImage blackKnight;
```

```
ofImage whiteRook;
ofImage blackRook;
ofImage whiteBishop;
ofImage blackBishop;
ofImage whiteKing;
ofImage blackKing;
ofImage whiteQueen;
ofImage blackQueen;
```

After loading them all as it says in the openframeworks documentation:

$$whitePawn.load(\text{"}wpawn.png\text{"});$$

(provided "wpawn.png" is in the bin/data folder of the project) I must create a new function to draw the pieces based on the board state:

```
for (int i = 0; i < n; i++)
{
   for (int j = 0; j < n; j++)
   {
      int piece = board.board[i][j];
      if (piece == 0) continue;
      ofRectangle info = BORDRECTS[n*(i) +j];
      switch (piece)
      {
      case White+Pawn:
         whitePawn.draw(info.x,info.y,squareLen,squareLen);
         break;
      case White+Knight:
         whiteKnight.draw(info.x, info.y, squareLen, squareLen);
         break;
      case White+Bishop:
         whiteBishop.draw(info.x, info.y, squareLen, squareLen);
         break;
      case White+Rook:
         whiteRook.draw(info.x, info.y, squareLen, squareLen);
         break;
      case White+Queen:
         whiteQueen.draw(info.x, info.y, squareLen, squareLen);
         break;
   //and so on

      }
```

```
    }
}
```

And with that, we have chess !

2.5 The Board Class

The board object is not just a matrix in the ofApp, but an instance of a higher board class. The over 500 line class is summarized in this UML diagram:

```
+-----------------------------------+
|            <<class>>              |
|             Board                 |
+-----------------------------------+
| -CASTLING: std::string //eg: KQkq, shows where you can castle
| -player: int //int representing who's turn it is. either 8 or 16
| -MOVES: int //total number of moves made
| -HALFMVES: int //total number of half moves, useful for 50 move rule
| -state: int //whether the game is over, or and of he end conditions have been
   reached
| -FORCED: int //en passant square, index either -1 if no square is available or
   in [16,23]  [40, 47]
| -tempBoard: int[8][8] //another matrix object used for theorizing on move
   results
| -kingMoved: bool[2] //bool tracking whether either king has moved, used for
   castling
| -rookMoved: bool[4] //bool tracking whether any rook has moved, used for
   castling
| +board: int[8][8] //board object representing where pieces currently are
| +promoFlag: int //variable representing if player can promote
```

```
| +lastMoveIndex: int //index of last move in [0,63]
| +dangerSquaresLen: int //amount of square the enemy controls
| +legalLen: int //amount of squares player can move to
| +checkSq: int //square the king in check is on in [0,63]
| +dangerSquares: int[128] //array of squares that the enemy can capture on
| +legalSquares: int[128] //array of squares player can move on
+----------------------------------+
| +Board()                         |
| +updateTempRep(): void `1        |
| +swapSides(): void               |
| +printBoard(int[8][8]): void     |
| +checkSquared(int[8][8]): int    |
| +setBoard(): void                |
| +updateTheoreticalCheck(int, int, int): void |
| +pinned(int, int, int): bool     |
| +move(int*, int*): void          |
| +moveAIa(int): void              |
| +promote(): void                 |
| +owariDa(): std::string //returns message if game is over
| +setState(int): void             |
| +getPlayer(): int                |
| +getHalfMoves(): int             |
| +getMoves(): int                 |
| +legalSquare(int[8][8], bool): void |
| +possSquares(int[8][8]): void |
| -removeCastlingRight(char): void |
| -legal(int[8][8], int, int, int, int): bool |
| -legalPawn(int[8][8], int, int, int, int, int, bool): bool |
| -legalKnight(int[8][8], int, int, int, int, int): bool |
| -legalBishop(int[8][8], int, int, int, int, int): bool |
| -legalRook(int[8][8], int, int, int, int, int): bool |
| -legalQueen(int[8][8], int, int, int, int, int): bool |
| -legalKing(int[8][8], int, int, int, int, int, bool): bool |
| -updateCastling(char): void      |
+----------------------------------+
```

2.6 Moving the Pieces

Well, not quite. Chess is an interesting programming challenge mostly because of the numerous rules that you must implement in your program, and make work together. When we go to move a piece to another square, we must check
1. The selected square has a piece, 2. The piece is on the current side's team, 3. The destination square does not have a piece on the current side's team, and 4. The move does not expose the king to danger. To do this I wrote a move function, that is called right after the select function in mousePressed. It checks those first three things in its guard clauses

```
void move()
{
   if (!gameContinue) return;
   if (AI && selectIndex != -1 && destIndex != -1 &&
      piecesTeam(board.board[selectIndex / 8][selectIndex % 8]) != White) return;
   board.move(&selectIndex, &destIndex);
    board.checkSq = board.checkSquared(board.board);
}
//in Board
void move(int * selectIndex, int * destIndex)
{
    //if (checkSq != -1) return;
    updateTempRep();
    if (promoFlag > Black) return; //
    if (*destIndex == -1 || *selectIndex == -1) return;//if no destination or
        source, return
    int piece = board[*selectIndex / 8][*selectIndex % 8];//piece at selected
        square
    if (piece == None) { *destIndex = -1; return; } //if no piece at selected
        square, return
    if (*destIndex != FORCED) FORCED = -1;//if not en passant, reset forced
    bool legalMove = legal(board, piece, player, *selectIndex, *destIndex);
    if (!legalMove)
    {
        *destIndex = -1;
        *selectIndex = -1;
        return;//if not legal move, return
    }
    bool pinn = pinned(*destIndex, *selectIndex, piece);
    if (pinn)
    {
      *destIndex = -1;
      *selectIndex = -1;
      return;//if not pinned, return
   }
    if (checkSq == -1)//if legal move‘
    {
        board[*destIndex / 8][*destIndex % 8] = piece;//move piece
        board[*selectIndex / 8][*selectIndex % 8] = None;//remove piece from old
            square

   }
    *selectIndex = -1;
    *destIndex = -1;
   // printBoard(tempBoard);
}
```

This shortened version of the move function shows the main functionality of the board class's move function: copying a piece to a square and then deleting it from its orignal square by setting it equal to None, and then reset the destination and select index back to -1 (off the board) via a pointer to that variable. The only part I've yet to explain is the legal function. It takes in the board, piece to move, side who's moving, and the select and destination index and returns whether or not the move would be allowed. The legal function first determines what piece you are moving, and points it toward its individual legal function as outlined in the UML diagram. For the sliding pieces (rook, bishop, queen), it checks if there is a piece in the way of the destination, and if the destination is in a straight line, like in the following snippets:

```
bool legalRook(int board[8][8], int side, int srcRank, int srcFile, int dstRank,
    int dstFile)
{
   if (srcRank != dstRank && srcFile != dstFile) return false;//straight check
    //determines direction then iterates though board to see if there is anything
        in between
    int xDir = dstFile - srcFile;
    int yDir = dstRank - srcRank;
    bool vertical = false;
    if (xDir == 0) vertical = true;
    int xsign = 0;
    int ysign = 0;
    if (vertical) ysign = yDir / abs(yDir);
    else xsign = xDir / abs(xDir);
    int itVar = abs(yDir) + abs(xDir);
    for (int i = 1; i < itVar; i++)
    {
        int x = srcFile + i * xsign;
        int y = srcRank + i * ysign;
        if (board[y][x] != None) return false;
    }
}
bool legalBishop(int board[8][8], int side, int srcRank, int srcFile, int
    dstRank, int dstFile)
{
   int xDir = dstFile - srcFile;
    int yDir = dstRank - srcRank;
    if (abs(xDir) != abs(yDir)) return false;//diagonal check
    //does the same, determines direction then iterates though board to see if
        there is anything in between. if so, return false
}
bool legalQueen(int board[8][8], int side, int srcRank, int srcFile, int
    dstRank, int dstFile)
{
   return legalRook(board, side, srcRank, srcFile, dstRank, dstFile) ||
```

```
    legalBishop(board, side, srcRank, srcFile, dstRank, dstFile) ;
}
```

The knight's legality function is staggeringly simple, it just checks if you are moving 2 ranks and 1 file, or vice versa. And similarly the kings legal move function checks if the change in rank and file are both equal to 1 at most.

2.7 Finishing Touches

For the most part, we now have chess! The only thing is that I have glossed over some details like checks/pins, castling, en passant, and the lose conditions. For more details, feel free to check my source code for the boring implementation details, though keep in mind I had to basically start the project over so some things are not optimally implemented.

Checks/Pins In the legal Move function, there is a boolean called pinn, equal to the output of a similarly named function. While the definition of the function is available in section 4, I can illustrate just what is is that function is doing. First, it checks defines some helpful variables before checking if the move in question is an en passant capture. The reason it does this is because of a special edge case where the function must return true: the the en passant would expose your king to danger, the function needs to return true. Next it does the move on the tempboard, then calculates all the squares the enemy hits. If the king is on one of them, it returns true.

Castling By saving whether or note the kings or rooks have moved, you can add an exception to the king's legal check for castling. This allows for the king to castle if that's it and the rook's first move.

En Passant Simply put, if a pawn moves two squares forward (only allowed on its first move) it can be the target of an en passant capture if there is an enemy pawn besides it. By saving the square behind this pawn, you add a legal square for an adjacent pawn to capture on.

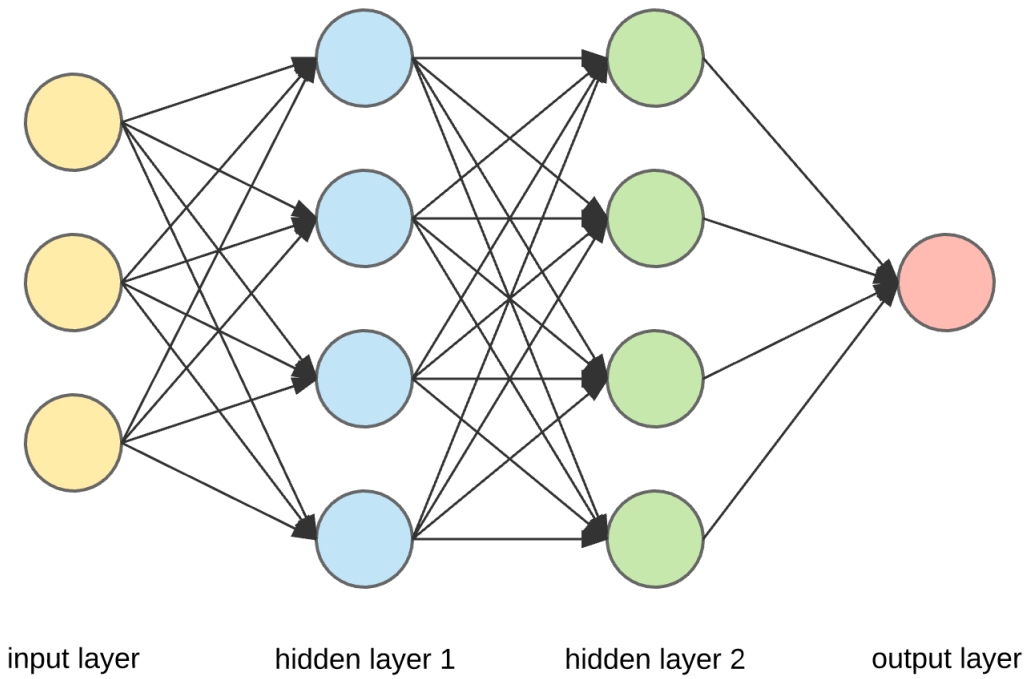
End Conditions Each turn, all possible moves that you can make are generated. If there is

no moves available, then the game is stalemate. If there are no move and you are in check, then you are checkmated. Two other end conditions are implemented: 50 move rule and insufficient material. While there isn't a queen or a pawn on the board, then the game adds up your pieces each move. If they dont add up to a Rook (4), then its a draw by insufficient material. This works since a knight is represented as 2 and a bishop is 3. two knights or a bishop and a knight both add up to a rook, but either piece by itself does not. The game shows a message describing why it ended:

2.8 StockBear

Great, now we have chess, a lovely game to play with friends. However, friends were not included in the openframeworks library, so we are forced to play by ourselves. Unless, we create an opponent. Using something like Stockfish is technically beyond the scope of what we are allowed to use in this project, but mimicking it is not, funnily enough. I am of course alluding to a neural network, the bread and butter of artificial intelligence. The basic idea behind a neural network is as follows: you have multiple lists of nodes that store values called

layers, you have one layer for the input, one for output, and layers in between for calculation called hidden layers. Visually, a neural network looks like this:



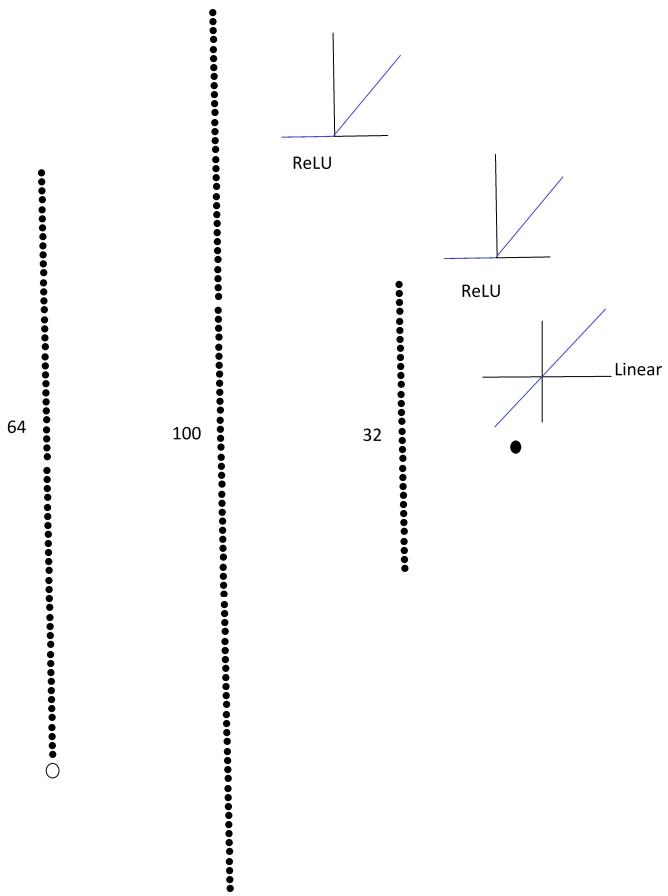
Each node of input is interconnected to every neuron in the first hidden layer by a number known as a weight, and each neuron also has a number called a bias. So for a given ith neuron in the first hidden layer, its neuron value is

$$a_i = (\sum_{j=1}^{n} w_{ij} * i_j) + b_i$$

for all neurons in the hidden layer, and an input of j features. After multiplying it by this, you add the bias and then apply an activation function. The point of an activation function is so that the final output isnt just an affine/linear transformation of the input. Adding this non linearity allows the neural network to approximate all kinds of functions, any function for that matter. So the final value for the neurons in the hidden layer is

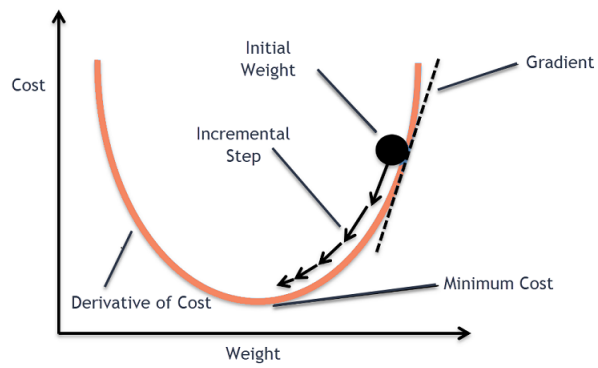
$$z_i = f(a_i)$$

. You repeat this for the next hidden layer, and the output layer, but since my goal is to get a continuous output (regression), I won't be applying an activation function to the output. Since my goal is to mimick Stockfish's output using regression, I will name my model StockBear (when a stock 'regresses' you have a "bear" market, Stockfish + regression = StockBear). StockBears architecture is as follows: 65 node input layer, 64 of those nodes representing the chess board via the integers described in 2.3, and then one more node for the turn. I included the turn node so that the final result wouldn't just sacrafice its queen every chance it gets, hopefully learning some sense of "this is a bad move"; 100 nodes in the first hidden layer. Here StockBear expands the input, hopefully drawing some connections between the representation of the board and the placement of the pieces; The next is a hidden layer of 32 nodes, here the information is contracted heavily; finally there is one node for the output which represents StockBear's appraisal. Here is a visual representation of StockBear's architecture that also shows the non linear activation functions:



Keep in mind, all layers are interconnected in a matrix of weight and biases.

This is a fine and dandy model, but how can this thing hope to approach Stockfish's excellence? Well via a technique known as gradient descent. By giving it inputs we know the expected value of, we can adjust the weights and biases to in the future return that expected value. This concept is carried by the concept of a loss function. By calculating the error of the model for a given input, or the average error for a batch of inputs, we can get a function who's inputs are our weights and biases, aka parameters, and who's output is the error. By finding a local minimum of this function and adjusting our weights and biases to this point, out network can become better at the task at hand. Provided is a one dimensional analog of what we are doing across our tens of thousand of variables:



By computing the gradient of the multivariable cost/loss funciton, we find the direction its increasing in the fastest. If we step all of our parameters in the opposite direction, the cost will descrease as a result. So by getting the derivative of the cost function in terms of our parameters, we learn how to adjust them to lower the error. By initializing the model with random weights, then performing gradient descent with stockfish appraisals, we can get a simple version of Stockfish to play against. I did this via the file at the end of the report. But here are some examples from my implementation of gradient descent:

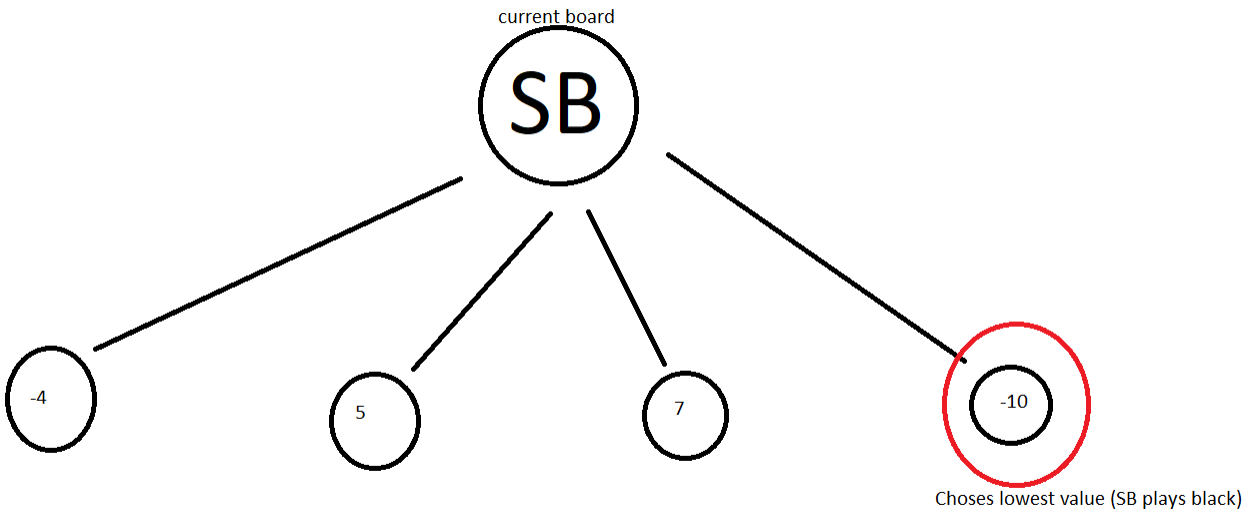
$$J = S(i) - Y$$

cost function for linear regression (just a difference of predicted and actual)

$$\delta W_{ij} = \eta * J * z_i$$

The gradient of the weights in the output layer is eta (arbitrary hyper-parameter that controls how fast the model learns) times the cost times the activation in the second hidden layer (via the chain rule). When you continue this pattern for the other layers, and keep the chain rule in mind, you will get how you must change the each parameter to better converge on a minimum of loss So for stockbear's algorithm, it simply looks at all possible moves by iterating over the board squares, and checking if its pieces can move to any other square via the legal function, then appraises the board if that were to happen. It then chooses the

board with the best appraisal according to its output.



StockBear in its current form was trained on 1500 different boards. The source code detailing SB's conception is at the end of the report. And with that we have an AI opponent!

3 Testing and Output

And just how does StockBear perform, lets see!

Well at least it started out ok. Making a computer decent at chess is harder than expected...

3.1 Notes

A short addendum to address things not covered in paper:

· The board was represented in training the same as it was in the game, as an array of integers that are the pieces. This is a terrible approach for machine learning, as though a computer sees very little similarity in the numbers 11 and 10, mathematically they are very close on the ranges we work with. This is significant because a neural network is a mathematical model, and the poor board representation led to poor convergence.

· A proper representation I could have used to differentiate the pieces for the model was one hot encoding. However, this would have made the input space increase by a factor of 7 (1 bit for color, and 6 bits that can represent the piece), so I took a hit on reliability to have parity with the efficient board representation used in game.

· The assets were ripped from chess.com

4 Code

HELPER.cpp

```
#include "ofApp.h"
#include "ofMain.h"
#include "stockbear.h"
#include <cstdlib>
#include <math.h>


const int None = 0;
const int Pawn = 1;
const int Knight = 2;
const int Bishop = 3;
const int Rook = 4;
const int Queen = 5;
const int King = 6;
const int White = 16;
const int Black = 8;
string PieceDictString = "xpnbrqk";
string PieceDicUpper = "XPNBRQK";

extern const int n = 8;

bool inArr(int n, int arr[], int x)
{
    for (int i = 0; i < n; i++)
    {
      if (arr[i] == x) return true;
   }
   return false;
}


float ReLU(float x) { return max(float(0), x); };
float appraiseAI(int board[][8])
{
    float inp[65];
    for (int i = 0; i < 64; i++)
    {
        inp[i] = (float)board[i / 8][i % 8];
    }
    inp[64] = float(0);
    float h1neurons[100];
```

```
    for (int i = 0; i < 100; i++)
    {
        h1neurons[i] = 0;
        for (int j = 0; j < 65; j++)
        {
            h1neurons[i] += inp[j] * i_h1[j][i];
        }
        h1neurons[i] += h1B[i];
        h1neurons[i] = ReLU(h1neurons[i]);
    }
    float h2neurons[32];
    for (int i = 0; i < 32; i++)
    {
        h2neurons[i] = 0;
        for (int j = 0; j < 100; j++)
        {
            h2neurons[i] += h1neurons[j] * h1_h2[j][i];
        }
        h2neurons[i] += h2B[i];
        h2neurons[i] = ReLU(h2neurons[i]);

    }
    float output = 0;
    for (int i = 0; i < 32; i++)
    {
        output += h2neurons[i] * h2_o[i];
    }
    output += outBias;
    return output;
}
void removeDuplicates(int* n, int arr[])
{
    int len = *n;
   int newArr[64];
   int newArrLen = 0;
    for (int i = 0; i < len; i++)
    {
      bool duplicate = false;
        for (int j = 0; j < newArrLen; j++)
        {
         if (arr[i] == newArr[j]) duplicate = true;
      }
        if (!duplicate)
        {
         newArr[newArrLen] = arr[i];
         newArrLen++;
      }
```

```
   }
    for (int i = 0; i < newArrLen; i++)
    {
      arr[i] = newArr[i];
   }
   *n = newArrLen;

}
int minimum(int n, float tings[])
{
    int min = INT_MAX;
    int inde = 0;
    for (int i = 0; i < n; i++)
    {
        if (tings[i] < min) {
            min = tings[i]; inde = i;
        }
    }
    return inde;
}

int otherTeam(int team)
{
    return White - (team - Black);
}

int piecesTeam(int piece)
{

   return piece > White ? White : piece == None ? None : Black; //alright maybe
       c++ is kinda based
}
int pieceType(int piece)
{
    return piece - piecesTeam(piece);
}
int pieceTypeEnemy(int piece)
{
    return piece - otherTeam(piecesTeam(piece));
}
int findChar(string str, char c)
{
    for (int i = 0; i < str.length(); i++)
    {
      if (str[i] == c) return i;
   }
   return -1;
```

```
}
int pieceToChar(int piece)
{
   int side = piecesTeam(piece);
   int type = pieceType(piece);
   char c = PieceDictString[type];
   if (side == White) c = toupper(c);
   return c;
}
const int spaces_until_info = 2;//for back when I supported fen....
const int startPieces[n] = { Rook,Knight,Bishop,Queen,King,Bishop,Knight,Rook };
class Board
{

    string CASTLING;
    int player;
    int MOVES;
    int HALFMVES;
    int state;
    int FORCED; //for en passant, index of board from h8 to a1, can only be one
        value at a time, so using int representation of index
    int tempBoard[8][8]; //for check checking
    bool kingMoved[2] = { false,false };
    bool rookMoved[4] = { false,false,false,false };
public:
    int board[n][n];
    int promoFlag;
    int lastMoveIndex;
    int dangerSquaresLen = 0;
    int legalLen = 0;
    int checkSq = -1;
    int dangerSquares[128]; // larger just incase too many duplicate threatenened
        squares caused segmentation fault
    int legalSquares[128];
    Board()
    {
        setBoard();
    }
    void updateTempRep()
    {
        for (int i = 0; i < 8; i++)
        {
            for (int j = 0; j < 8; j++)
            {
                tempBoard[i][j] = board[i][j];
            }
        }
```

```
}
void swapSides()
{
  player = otherTeam(player);
}
void printBoard(int x[n][n])
{
    for (int j = 0; j < n; j++)
    {
        for (int i = 0; i < n; i++)
        {
            if (inArr(dangerSquaresLen, dangerSquares, j* n + i))
                printf("%3dx", x[j][i]);
            else printf("%4d", x[j][i]);
        }
        cout << endl;
    }
    cout << endl;
}
int checkSquared(int bord[8][8])
{
   for (int i = 0; i < dangerSquaresLen; i++)
   {
      if (bord[dangerSquares[i] / 8][dangerSquares[i] % 8] == King + player)
         return dangerSquares[i];
  }
  return -1;
}
void setBoard()
{
    promoFlag = -1;
    CASTLING = "KQkq";
    player = White;
    MOVES = 1;
    HALFMVES = 0;
    state = 5;
    FORCED = -1;
    dangerSquaresLen = 0;
    checkSq = -1;
    legalLen = 0;
    for (int i = 0; i < 4; i++)
    {
        kingMoved[i / 4] = false;
        rookMoved[i] = false;
    }
    for (int i = 0; i < 8; i++)
    {
```

```
        board[1][i] = Pawn + Black;
        board[6][i] = Pawn + White;
    }
    for (int i = 0; i < 8; i++)
    {
     board[0][i] = startPieces[i] + Black;
     board[7][i] = startPieces[i] + White;
  }
    for (int i = 2; i < 6; i++)
    {
        for (int j = 0; j < 8; j++)
        {
            board[i][j] = None;
        }
    }
    updateTempRep();
}
void updateTheoreticalCheck(int destIndex, int selectIndex, int piece)
{
    //updates checkSq to the value it would be if the move was made
    tempBoard[destIndex / 8][destIndex % 8] = piece;//move piece
    tempBoard[selectIndex / 8][selectIndex % 8] = None;//remove piece from
        old square
    player = otherTeam(player);//switches sides
    possSquares(tempBoard);
    player = otherTeam(player);
    checkSq = checkSquared(tempBoard);
    player = otherTeam(player);
}
bool pinned(int destIndex, int selectIndex, int piece)
{
    //return true if move would expose king to danger
    int okabeSq = -1; //hypothetical checkSq
    int dRank = (destIndex/8) - (selectIndex/8);
    bool enPassant = pieceType(piece) == Pawn &&
        piecesTeam(tempBoard[(destIndex / 8) - dRank][destIndex % 8]) ==
        otherTeam(player);
    if (enPassant) //only true when capturing en passant
    {
        tempBoard[(destIndex / 8) - dRank][destIndex % 8] =
            None;//technically anything can be captured enpassant, but only
            pawns are able to change EP info
    }
    tempBoard[destIndex / 8][destIndex % 8] = piece;//move piece
    tempBoard[selectIndex / 8][selectIndex % 8] = None;//remove piece from
        old square
    //printBoard(tempBoard);
```

```
    player = otherTeam(player);//switches sides
    possSquares(tempBoard);
    player = otherTeam(player);
    okabeSq = checkSquared(tempBoard);
    if (enPassant && okabeSq != -1) //if move exposes king and was en
        passant, undo enpassant
    {
        board[(destIndex / 8) - dRank][destIndex % 8] = Pawn +
            otherTeam(player);//the rebooted one
    }
    updateTempRep();
    return okabeSq == -1 ? false : true;
}
void move(int * selectIndex, int * destIndex)
{
    //if (checkSq != -1) return;
    updateTempRep();
    if (promoFlag > Black) return; //
    if (*destIndex == -1 || *selectIndex == -1) return;//if no destination or
        source, return
    int piece = board[*selectIndex / 8][*selectIndex % 8];//piece at selected
        square
    if (piece == None) { *destIndex = -1; return; } //if no piece at selected
        square, return
    if (player == None) return;//if not player's turn, return (changed to
        NONE because I'm not implementing side check, if you can move in the
        middle of the opponent's turn, you got it)
    if (*destIndex != FORCED) FORCED = -1;//if not en passant, reset forced
    bool legalMove = legal(board, piece, player, *selectIndex, *destIndex);
    if (!legalMove)
    {
        *destIndex = -1;
        *selectIndex = -1;
        return;//if not legal move, return
    }
    bool pinn = pinned(*destIndex, *selectIndex, piece);
    if (pinn)
    {
     *destIndex = -1;
     *selectIndex = -1;
     return;//if pinned, return
  }
    if (checkSq == -1)//if legal move‘
    {
        if (pieceType(piece) == Pawn || pieceType(board[*destIndex /
            8][*destIndex % 8]) != None) HALFMVES = 0;//if pawn moves or
            capture, reset halfmove clock
```

```
    else HALFMVES++; //if you dont capture, increment halfmove clock
       board[*destIndex / 8][*destIndex % 8] = piece;//move piece
       board[*selectIndex / 8][*selectIndex % 8] = None;//remove piece from
          old square
       if (piece == Pawn + player && (*destIndex / 8) % 7 == 0) promoFlag =
          2;//if pawn reaches end, promote to queen
       if (pieceType(piece) == Rook)
       {
          int srcFile = *selectIndex % 8;
          if (srcFile == 0 && player == Black) rookMoved[0] = true;
          else if (srcFile == 7 && player == Black) rookMoved[1] = true;
          else if (srcFile == 0 && player == White) rookMoved[2] = true;
          else if (srcFile == 7 && player == White) rookMoved[3] = true;
       }
       if (pieceType(piece) == King)
       {
       if (player == White) kingMoved[0] = true;
       else kingMoved[1] = true;
    }
       //pinn();
       possSquares(board);//update possible square the opp hits, TODO:
          overload to show legal move for selected piece
       player = otherTeam(player);//switches sides
       lastMoveIndex = *destIndex;
       if (player == Black) MOVES++;
       updateTempRep();

  }
    else//if legal move‘
    {
       updateTheoreticalCheck(*destIndex, *selectIndex, piece); //TODO: USE
          pinned HERE
       if (checkSq == -1)
       {
          player = otherTeam(player);
          move(selectIndex, destIndex);
       }
       else
       {
          updateTempRep();
          player = otherTeam(player);
       }
    }
    *selectIndex = -1;
    *destIndex = -1;
}
void moveAIa(int side)
```

```
{
    legalSquare(board, true);
}
void promote()
{
    board[lastMoveIndex / 8][lastMoveIndex % 8] = promoFlag;
    promoFlag = -1;

}
string owariDa()
{
    if (state != 5)
    {
        string message;
        if (state == 0) message = player == White ? "CHECKMATE: BLACK WINS" :
            "CHECKMATE: WHITE WINS";
        else if (state == 1) message = "STALEMATE";
        else if (state == 2) message = "DRAW: INSUFFICIENT MATERIAL";
        else if (state == 3) message = "DRAW: FIFTY MOVE RULE";
        else if (state == 4) message = "DRAW: THREEFOLD REPETITION";
        return message;
    }
    return "";
}
void setState(int _state)
{
  state = _state;
}

int getPlayer()
{
    return player;
}
int getHalfMoves()
{
    return HALFMVES;
}
int getMoves()
{
  return MOVES;
}
void legalSquare(int bord[8][8], bool ai = false)
{
    legalLen = 0;
    int slct[64];
    float appraisals[64];
    int slcts = 0;
```

```
for (int i = 0; i < 8; i++)
{
    for (int j = 0; j < 8; j++)
    {
        if (piecesTeam(bord[i][j]) != player) continue;
        for (int k = 0; k < 64; k++)
        {
            if (pieceType(bord[i][j]) != Pawn && pieceType(bord[i][j]) !=
                King && legal(bord, bord[i][j], player, 8 * i + j, k) &&
                !pinned(k, 8 * i + j, bord[i][j]))//FASTER METHOD: create
                movement mask for each piece, and check which of those
                squares are legal
            {
                legalSquares[legalLen] = k;
                slct[slcts] = 8 * i + j;
                tempBoard[k / 8][k % 8] = bord[i][j];
                tempBoard[i][i] = None;
                appraisals[slcts] = appraiseAI(tempBoard); //make move n
                    tepboard and store thoughts
                slcts++;
                legalLen++;
            }
            if (pieceType(bord[i][j]) == King)
            {
                int dstRank = k / 8;
                int dstFile = k % 8;
                int opp = bord[dstRank][dstFile];

                if (piecesTeam(opp) == player) continue;
                if (legalKing(bord, player, i, j, dstRank, dstFile, false)
                    && !pinned(k, 8 * i + j, bord[i][j]))//FASTER METHOD:
                    create movement mask for each piece, and check which of
                    those squares are legal
                {
                    legalSquares[legalLen] = k;
                    slct[slcts] = 8 * i + j;
                    tempBoard[k / 8][k % 8] = bord[i][j];
                    tempBoard[i][i] = None;
                    appraisals[slcts] = appraiseAI(tempBoard); //make move
                        n tepboard and store thoughts
                    slcts++;
                    legalLen++;
                }

            }
            if (pieceType(bord[i][j]) == Pawn)
            {
```

```
                int dstRank = k / 8;
                int dstFile = k % 8;
                int opp = bord[dstRank][dstFile];

                if (piecesTeam(opp) == player) continue;
                if (legalPawn(bord, player, i, j, dstRank, dstFile, false)
                    && !pinned(k, 8 * i + j, bord[i][j]))//FASTER METHOD:
                    create movement mask for each piece, and check which of
                    those squares are legal
                {
            legalSquares[legalLen] = k;
                    slct[slcts] = 8 * i + j;
                    tempBoard[k / 8][k % 8] = bord[i][j]; //i aint
                        implementing en passant
                    tempBoard[i][i] = None;
                    appraisals[slcts] = appraiseAI(tempBoard); //make move
                        n tepboard and store thoughts
                    slcts++;
            legalLen++;
          }

            }
        }
        updateTempRep();
    }
}
if (ai)
{
    int BestMoveQuotes = minimum(slcts, appraisals); //index of "best
        move"
    int slc = slct[BestMoveQuotes];
    int dst = legalSquares[BestMoveQuotes];
 move(&slc, &dst);
}
removeDuplicates(&legalLen, legalSquares);
}

void possSquares(int bord[8][8])
{
    dangerSquaresLen = 0;
    for (int i = 0; i < 8; i++)
    {
        for (int j = 0; j < 8; j++)
        {
            if (piecesTeam(bord[i][j]) != player) continue;
            for (int k = 0; k < 64; k++)
            {
```

```
                if (pieceType(bord[i][j]) != Pawn && legal(bord, bord[i][j],
                    player, 8 * i + j, k))//FASTER METHOD: create movement mask
                    for each piece, and check which of those squares are legal
                {
                    dangerSquares[dangerSquaresLen] = k;
                    dangerSquaresLen++;
                }
                if (pieceType(bord[i][j]) == Pawn)
                {
                    bool offByOne = (k % 8 == j + 1 || k % 8 == j - 1);
                    if (!offByOne) continue;
                    bool upOrDownByOne = ((player == White && k / 8 == i - 1)
                        || (player == Black && k / 8 == i + 1));
                    if (!upOrDownByOne) continue;
                    dangerSquares[dangerSquaresLen] = k;
                    dangerSquaresLen++;
                }
            }

        }
    }
    removeDuplicates(&dangerSquaresLen, dangerSquares);
}
private:

    void removeCastlingRight(char c)
    {
        string newCastling = "";
        for (int i = 0; i < CASTLING.length(); i++)
        {
            if (CASTLING[i] != c) newCastling += CASTLING[i];
        }
        CASTLING = newCastling;
    }
    bool legal(int board[8][8], int piece, int side, int src, int dst)
    {
        piece = piece - side;
        bool movingWhite = board[src / 8][src % 8] > White;
        if ((side == White && !movingWhite) || (side == Black && movingWhite)) {
            cout << "not ur turn" << endl; return false; }
        int dstRank = dst / 8;
        int dstFile = dst % 8;
        int opp = board[dstRank][dstFile];
        bool backDoor = ((side==White)&&(opp>White)) ||
            ((side==Black)&&(opp>Black)&&(opp<White));
        if (backDoor) return false;
        int srcRank = src / 8;
```

```
    int srcFile = src % 8;
    if (piece == Pawn) return legalPawn(board, side, srcRank, srcFile,
        dstRank, dstFile);
    if (piece == Knight) return legalKnight(board, side, srcRank, srcFile,
        dstRank, dstFile);
    if (piece == Bishop) return legalBishop(board, side, srcRank, srcFile,
        dstRank, dstFile);
    if (piece == Rook) return legalRook(board, side, srcRank, srcFile,
        dstRank, dstFile);
    if (piece == Queen) return legalQueen(board, side, srcRank, srcFile,
        dstRank, dstFile);
    if (piece == King) return legalKing(board, side, srcRank, srcFile,
        dstRank, dstFile);
}
bool legalPawn(int board[8][8], int side, int srcRank, int srcFile, int
    dstRank, int dstFile, bool unironicallyMoving = true)
{
    int dRank = dstRank - srcRank;
  int dFile = dstFile - srcFile;
  int opp = board[dstRank][dstFile];
    if (side == White)
    {
     if (dRank == -1 && dFile == 0 && opp == None) return true;
     if (dRank == -1 && abs(dFile) == 1 && opp > Black && opp < White) return
         true;
        if (dRank == -2 && dFile == 0 && srcRank == 6 && opp == None &&
            board[5][srcFile] == None) {
            if (unironicallyMoving) FORCED = 8*(dstRank+1) + dstFile;
            return true;
        }
  }
    if (side == Black)
    {
     if (dRank == 1 && dFile == 0 && opp == None) return true;
     if (dRank == 1 && abs(dFile) == 1 && opp > White) return true;
     if (dRank == 2 && dFile == 0 && srcRank == 1 && opp == None &&
         board[2][srcFile] == None) {
            if (unironicallyMoving) FORCED = 8 * (dstRank - 1) + dstFile;
            return true;
        }
  }
  if (abs(dRank) == 1 && abs(dFile) == 1 && dstRank == FORCED / 8 && dstFile
      == FORCED % 8)
    {
        board[(FORCED / 8)-dRank][FORCED % 8] = None;//technically anything
            can be captured enpassant, but only pawns are able to change EP
            info
```

```
        FORCED = -1;
        return true;
    }
  return false;
}
 bool legalKnight(int board[8][8], int side, int srcRank, int srcFile, int
    dstRank, int dstFile)
 {
    int dRank = dstRank - srcRank;
    int dFile = dstFile - srcFile;
    if (abs(dRank) == 2 && abs(dFile) == 1) return true;
    if (abs(dRank) == 1 && abs(dFile) == 2) return true;
    return false;
 }
 bool legalBishop(int board[8][8], int side, int srcRank, int srcFile, int
    dstRank, int dstFile)
 {
  int xDir = dstFile - srcFile;
    int yDir = dstRank - srcRank;
    if (abs(xDir) != abs(yDir)) return false;//diagonal check
    int xsign = xDir / abs(xDir);
    int ysign = yDir / abs(yDir);
    for (int i = 1; i < abs(xDir); i++)
    {
     int x = srcFile + i * xsign;
     int y = srcRank + i * ysign;
     if (board[y][x] != None) return false;
  }
    return true;
}
 bool legalRook(int board[8][8], int side, int srcRank, int srcFile, int
    dstRank, int dstFile)
 {
  if (srcRank != dstRank && srcFile != dstFile) return false;//straight check
    int xDir = dstFile - srcFile;
    int yDir = dstRank - srcRank;
    bool vertical = false;
    if (xDir == 0) vertical = true;
    int xsign = 0;
    int ysign = 0;
    if (vertical) ysign = yDir / abs(yDir);
    else xsign = xDir / abs(xDir);
    int itVar = abs(yDir) + abs(xDir);
    for (int i = 1; i < itVar; i++)
    {
        int x = srcFile + i * xsign;
        int y = srcRank + i * ysign;
```

```
        if (board[y][x] != None) return false;
    }
    return true;
}
bool legalQueen(int board[8][8], int side, int srcRank, int srcFile, int
    dstRank, int dstFile)
{
  return legalRook(board, side, srcRank, srcFile, dstRank, dstFile) ||
      legalBishop(board, side, srcRank, srcFile, dstRank, dstFile) ;
}
void updateCastling(char casl = 'n')
{
    if (kingMoved[0])
    {
        removeCastlingRight('Q');
        removeCastlingRight('K');
    }
    if (kingMoved[1])
    {
     removeCastlingRight('q');
     removeCastlingRight('k');
  }
    if (rookMoved[0]) removeCastlingRight('q');
    if (rookMoved[1]) removeCastlingRight('k');
    if (rookMoved[2]) removeCastlingRight('Q');
    if (rookMoved[3]) removeCastlingRight('K');
    if (casl == 'n') return;
  removeCastlingRight(casl);
}
bool legalKing(int board[8][8], int side, int srcRank, int srcFile, int
    dstRank, int dstFile, bool fr = true)
{
    updateCastling();
    int xDir = dstFile - srcFile;
    int yDir = dstRank - srcRank;
    bool lazyKing = side == White ? srcRank == 7 && srcFile == 4 : srcRank ==
        0 && srcFile == 4;
    if (abs(yDir) > 1) return false;
    if (yDir == 0 && lazyKing && abs(xDir) == 2)
    {
        if (checkSq != -1) return false;
        int xsign = xDir / abs(xDir);
        int rookFile = xDir == 2 ? 7 : 0;
        for (int i = 0; i < dangerSquaresLen; i++)
        {
        if ((srcRank * 8)+srcFile + xsign == dangerSquares[i]) return false;
            if ((srcRank * 8) + srcFile + 2*xsign == dangerSquares[i]) return
```

```
                false;
            if (rookFile == 0 && (srcRank*8)+srcFile + 3 * xsign ==
                dangerSquares[i]) return false;
        }
           int rookRank = side == White ? 7 : 0;
           char sideCastl = 'K';
           if (side == White) sideCastl = xsign == 1 ? 'K' : 'Q';
        else if (side == Black) sideCastl = xsign == 1 ? 'k' : 'q'; //k for
            upper left, q for upper right
           if (findChar(CASTLING, sideCastl) == -1) return false; //if you can
               castle return
           if (board[srcRank][rookFile] != Rook + side) return false; //can only
               castle with rook
           //fuck the for loop, doing this manually
           if(board[srcRank][srcFile + xsign] != None) return false; //sum in
               the way
           if (board[srcRank][srcFile + 2*xsign] != None) return false; // mmmm
           if (rookFile == 0 && board[srcRank][srcFile + 3 * xsign] != None)
               return false;
           if (fr)
           {
               board[rookRank][srcFile + (((7-rookFile) >> 2) + 3) * xsign] =
                   None; //I'm kinda god like at coding IMO
               board[rookRank][srcFile + xsign] = Rook + side;
           }
           //(writes optimized code for a line called once every 100 runs) "na
               I'm actually different"
           return true;
       }
       if (abs(xDir) > 1) return false;
       return true;
   }
};
```

ofApp.cpp

```
    #include "ofApp.h"
#include "ofMain.h"
#include <cstdlib>
#include <math.h>
#include "HELPER.cpp"
int DEBUG = 0;
//just gonna use n instead of literal 8, for no reason at all
Board board;
bool oneFold, twoFold;
int YDIS = 25;// WINHYT / 9;
int XDIS = 213;// WINLEN / 9;
bool AI = false;
```

```
ofColor BLACK(0, 0, 0);
ofColor WHITE(255, 255, 255);
ofColor Default[2] = { WHITE,BLACK };
ofColor LITEBROWN(196, 170, 137);
ofColor DARKBROWN(97, 68, 48);
ofColor WoodenTheme[2] = { LITEBROWN, DARKBROWN};
ofColor CHECKRED(174, 13, 45);
ofColor KINGCHECK(235, 30, 15);
int destIndex = -1;
ofRectangle TWOP = ofRectangle(WINLEN/3, WINHYT/3, WINLEN/3, 100);

ofRectangle BEARquotes = ofRectangle(WINLEN / 3, 2*WINHYT / 3, WINLEN / 3, 100);
ofRectangle BORDRECTS[64];

int boardWidth = WINLEN - 2*XDIS;
int squareLen = boardWidth / 8;

bool START = false;

ofImage whitePawn;
ofImage blackPawn;
ofImage whiteKnight;
ofImage blackKnight;
ofImage whiteRook;
ofImage blackRook;
ofImage whiteBishop;
ofImage blackBishop;
ofImage whiteKing;
ofImage blackKing;
ofImage whiteQueen;
ofImage blackQueen;
int selectIndex = -1;

ofImage blackPromoPieces[4];
ofImage whitePromoPieces[4];
string promo = "q";
bool gameContinue = true;

string GAMESTATE;


ofTrueTypeFont font;
/*16-(8-8) = 16
16-(16-8) = 8*/

void loadPNGS()
{
```

```
    font.loadFont("Analogist.ttf", 40);
    whitePawn.load("pieces/wpawn.png");
    blackPawn.load("pieces/bpawn.png");
    whiteKnight.load("pieces/wknight.png");
    blackKnight.load("pieces/bknight.png");
    whiteRook.load("pieces/wrook.png");
    blackRook.load("pieces/brook.png");
    whiteBishop.load("pieces/wbishop.png");
    blackBishop.load("pieces/bbishop.png");
    whiteKing.load("pieces/wking.png");
    blackKing.load("pieces/bking.png");
    whiteQueen.load("pieces/wqueen.png");
    blackQueen.load("pieces/bqueen.png");
    blackPromoPieces[0].load("pieces/bqueen.png");
    blackPromoPieces[1].load("pieces/bknight.png");
    blackPromoPieces[2].load("pieces/bbishop.png");
    blackPromoPieces[3].load("pieces/brook.png");
    whitePromoPieces[0].load("pieces/wqueen.png");
    whitePromoPieces[1].load("pieces/wknight.png");
    whitePromoPieces[2].load("pieces/wbishop.png");
    whitePromoPieces[3].load("pieces/wrook.png");
}
void drawPieces()
{
    ofSetColor(WHITE);
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                int piece = board.board[i][j];
                if (piece == 0) continue;
                ofRectangle info = BORDRECTS[n*(i) +j];
                switch (piece)
                {
                case White+Pawn:
                    whitePawn.draw(info.x,info.y,squareLen,squareLen);
                    break;
                case White+Knight:
                    whiteKnight.draw(info.x, info.y, squareLen, squareLen);
                    break;
                case White+Bishop:
                    whiteBishop.draw(info.x, info.y, squareLen, squareLen);
                    break;
                case White+Rook:
                    whiteRook.draw(info.x, info.y, squareLen, squareLen);
                    break;
                case White+Queen:
```

```
            whiteQueen.draw(info.x, info.y, squareLen, squareLen);
            break;
         case White+King:
            whiteKing.draw(info.x, info.y, squareLen, squareLen);
            break;
         case Black+Pawn:
            blackPawn.draw(info.x, info.y, squareLen, squareLen);
            break;
         case Black+Knight:
            blackKnight.draw(info.x, info.y, squareLen, squareLen);
            break;
         case Black+Rook:
            blackRook.draw(info.x, info.y, squareLen, squareLen);
            break;
         case Black+Bishop:
            blackBishop.draw(info.x, info.y, squareLen, squareLen);
            break;
         case Black+King:
            blackKing.draw(info.x, info.y, squareLen, squareLen);
            break;
         case Black+Queen:
            blackQueen.draw(info.x, info.y, squareLen, squareLen);
            break;

         }
      }
   }
}


void makeBoardDraw()
{
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         BORDRECTS[n * (i)+j].set(j * squareLen + (XDIS), i * squareLen + (YDIS /
            2), squareLen, squareLen);
      }
   }
}
void drawBoard()
{
   ofFill();
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
```

```
      {
         ofSetColor(WoodenTheme[((i + j) % 2)]);
         if (j + (n * i) == destIndex) ofSetColor(CHECKRED);
         else if (j + (n * i) == board.checkSq) ofSetColor(KINGCHECK);
         ofDrawRectangle(BORDRECTS[i * n + j]);
      }
   }
   if (board.dangerSquaresLen > 0 && DEBUG == 2)
   {
      ofSetColor(BLACK);
      for (int i = 0; i < board.dangerSquaresLen; i++)
      {
         ofDrawRectangle(BORDRECTS[board.dangerSquares[i]]);
      }
   }
   if (board.legalLen > 0 && DEBUG == 1)
   {
      ofSetColor(BLACK);
      for (int i = 0; i < board.legalLen; i++)
      {
         ofDrawRectangle(BORDRECTS[board.legalSquares[i]]);
      }
   }
}
void drawPromoBox()
{
   int boardEndPos = XDIS + 8 * squareLen;
   if (board.getPlayer() == Black) {
   for (int i = 0; i < 4; i++)
   {
      whitePromoPieces[i].draw(boardEndPos, YDIS/2 + squareLen*i, squareLen,
          squareLen);
   }
   }
   else
   {
      for (int i = 3; i >= 0; i--)
      {
         blackPromoPieces[3-i].draw(boardEndPos, (YDIS/2 + 7*squareLen) -
             (3-i)*squareLen, squareLen, squareLen);
      }

   }

}
int clampPiece(int yPos, int side)
{
```

```
   int pieceArray[4] = {Queen, Knight, Bishop, Rook};
   int piece = side == Black ? pieceArray[(yPos / squareLen)%4] :
      pieceArray[3-((yPos / squareLen) % 4)];
   return otherTeam(side) + piece;
}
void select(int x, int y)
{
   //TODO: Mathematical implementation that doesn't loop over board
   for (int i = 0; i < n * n; i++)
   {
      if (BORDRECTS[i].inside(x, y))
      {
         if (destIndex == i) destIndex = -1;//unaelect if they click selected
            square
         else
         {
            selectIndex = destIndex;
            destIndex = i;
         }
      }
   }
}
void print(int text)
{
   cout << text << endl;
}
void print(string text)
{
   cout << text << endl;
}
void setMatrixEqual(int inputMatrix[n][n], int copyFrom[n][n])
{
   for (int i = 0; i < n; i++)
   {
      for (int j = 0; j < n; j++)
      {
         inputMatrix[i][j] = copyFrom[i][j];

      }
   }
}
bool enoughMats()//0 if suff.
{
   int whitePieces = 0;
   int blackPieces = 0;
   for (int i = 0; i < 8; i++)
   {
```

```
      for (int j = 0; j < 8; j++)
      {
         int pieceS = board.board[i][j];
         if (pieceType(pieceS) == Pawn || pieceType(pieceS) == Queen) return true;
         if (pieceType(pieceS) == King) continue;
         if (piecesTeam(pieceS) == White) whitePieces += pieceType(pieceS);
         else blackPieces += pieceType(pieceS); //adds 0 if none
      }
   }
   if (whitePieces < Rook && blackPieces < Rook) return false;
   return true;
}
void move()
{
   if (!gameContinue) return;
   int moves = board.getMoves();
   if (AI && selectIndex != -1 && destIndex != -1 &&
       piecesTeam(board.board[selectIndex / 8][selectIndex % 8]) != White) return;
   board.move(&selectIndex, &destIndex);
   bool moveMade = board.getMoves() != moves;
   board.checkSq = board.checkSquared(board.board);
   board.legalSquare(board.board);
   if (board.legalLen == 0)
   {
      if (board.checkSq == -1)
      {
         board.setState(1);
      }
      else
      {
         board.setState(0);
      }
      gameContinue = false;
      GAMESTATE = board.owariDa();
   }
   else if (board.getHalfMoves() >= 50)
   {
      board.setState(3);
      gameContinue = false;
      GAMESTATE = board.owariDa();
   }
   else if (moveMade && !enoughMats())
   {
      board.setState(2);
      gameContinue = false;
      GAMESTATE = board.owariDa();
   }
```

```
}
void drawMenu()
{
   ofSetColor(38,8,138);
   ofDrawRectangle(0, 0, WINLEN, WINHYT);
   ofSetColor(WHITE);
   font.drawString("CHESS", WINLEN / 2 - 50, WINHYT / 4);
   ofDrawRectangle(TWOP);
   ofDrawRectangle(BEARquotes);
   ofSetColor(BLACK);
   font.drawString("2 player", TWOP.x+105, TWOP.y+55);
   font.drawString("AI Mode", BEARquotes.x + 105, BEARquotes.y + 55);
}
void startGame(int x, int y)
{
   if (TWOP.inside(x, y))
   {
      START = true;
   }
   else if (BEARquotes.inside(x, y))
   {
      AI = true;
      START = true;
   }
}
//-------------------------------------------------------------
void ofApp::setup()
{
   ofSetFrameRate(20);
   ofSetBackgroundColor(48,46,43);
   board.setBoard();
   makeBoardDraw();
   loadPNGS();
}

void ofApp::update()
{
   if (board.getPlayer() != White && AI && gameContinue)
   {
      board.moveAIa(board.getPlayer());
      selectIndex = -1;
      destIndex = -1;
   }
   if (board.promoFlag > Black)
   {
      int playerBeforePromo = board.getPlayer();
      board.promote();
```

```
      move();//update temp board
      board.swapSides();
      board.possSquares(board.board); //whites turf
      board.swapSides();
      board.checkSq = board.checkSquared(board.board);
      if (board.checkSq != -1) board.legalSquare(board.board);
      if (board.checkSq == -1) board.swapSides(); //TODO: automatic check
         detection after promotion
      board.promoFlag = -1;
      if (board.getPlayer() != playerBeforePromo) board.swapSides();
   }

}

void ofApp::draw()//every frame draw is called btw, duhh
{
   drawBoard();
   drawPieces();
   if (board.promoFlag != -1)
   {
      drawPromoBox();
   }
   if (!gameContinue)
   {
      board.getPlayer() == White ? ofSetColor(BLACK) : ofSetColor(WHITE);

      font.drawString(GAMESTATE, XDIS+squareLen, YDIS+4*squareLen);
   }
   if (!START)
   {
      drawMenu();
   }
}



//-------------------------------------------------------------
void ofApp::keyPressed(int key)
{
   if (key == OF_KEY_F4)
   {
      board.setBoard();
      gameContinue = true;
      oneFold = false;
      twoFold = false;
      START = false;
      AI = false;
```

```
    }
    else if (key == OF_KEY_F5)
    {
      DEBUG =  1;
    }
    else if (key == OF_KEY_F6)
    {
      DEBUG = 2;
    }
    else if (key == OF_KEY_F7)
    {
      DEBUG = 0;
    }
    else if (key == ' ')
    {
      destIndex = -1;
      selectIndex = -1;
      if (!gameContinue) {
        board.setBoard(); gameContinue = 1; }
    }

}

void ofApp::mousePressed(int x, int y, int button){
    if (!START)
    {
      startGame(x,y);
      return;
    }
    if (board.promoFlag == -1)
    {
      select(x, y);
      move();
    }
    else
    {
      y -= YDIS / 2;
      if (board.getPlayer() == White && y < squareLen * 4) return;
      if (board.getPlayer() == Black && y > squareLen * 4) return;
      if (x < XDIS + 8 * squareLen) return;
      board.promoFlag = clampPiece(y, board.getPlayer());
      return;
    }
}
```

model.cpp

```
    #include <iostream>
```

```
#include "evals.h" //huge header file storing the training data, 15000 pairs
    strings in the form .."[fen]","[appraisal]".. eg:
    "r2r4/1pk2npp/n1p1bp2/p1b1p3/2P1P3/PPN2PN1/1BK3PP/3RRB2 w - - 1 19","-0.51",
#include <fstream>
using namespace std;
const int inputDim = 65;
const int h1Dim = 100;
const int h2Dim = 32;
const int outputDim = 1;
const int None = 0;
const int Pawn = 1;
const int Knight = 2;
const int Bishop = 3;
const int Rook = 4;
const int Queen = 5;
const int King = 6;
const int White = 16;
const int Black = 8;
float random() { return 2*(rand()/float(RAND_MAX))-1; }
float random2() { return (rand()/float(RAND_MAX)); }

float dReLU(float x) {return x>float(0) ? float(1):float(0);}
void printArr(int n, float arr[])
{
   for (int i = 0; i < n; i++)
   {
      cout << arr[i] << " ";
   }
   cout << endl;
}
void printMatrix(int n, int m, float arr[][h1Dim])
{
   for (int j = 0;j<m;j++)
   {
      for (int i = 0; i < n; i++)
      {
         cout << arr[i][j] << " ";
      }
   }
   cout << endl;
}

void fenToVec(string fen, float vec[])
{
   int index = 0;
   for (int i = 0; i < fen.length(); i++)
   {
```

```
      char c = fen[i];
      if (c == '/') continue;
      if (c == ' ')
      {
         fen[i+1] == 'w' ? vec[index] = 1 : vec[index] = 0;
         break;
      }
      if (c>=49 && c<= 56) //c in between 1 and 8 inc (represents how many empty
          squares in fen)
      {
         for (int j = 0; j<(c-48); j++)
         {
            vec[index] = 0; // add that many blank squares
         }
         continue;
      }
      int team;
      int piece;
      team = isupper(c) ? White : Black;
      if (team == White) c = c+32;
      switch (c)
      {
         case 'p':
         piece = Pawn;
         break;
         case 'n':
         piece = Knight;
         break;
         case 'b':
         piece = Bishop;
         break;
         case 'r':
         piece = Rook;
         break;
         case 'q':
         piece = Queen;
         break;
         case 'k':
         piece = King;
         break;
      }
      vec[index] = (float)piece+team;
   }
}
class StockBear
{
public:
```

```
float error = 0;
float output[outputDim];
StockBear()
{
   srand(time(0));
   for (int i = 0; i < h1Dim; i++)
   {
      for (int j = 0; j < inputDim; j++)
      {
         h1weights[j][i] = random();
      }
      h1biases[i] = random();
      h1neurons[i] = 0;
   }
   for (int i = 0; i < h2Dim; i++)
   {
      for (int j = 0; j < h1Dim; j++)
      {
         h2weights[j][i] = random();
      }
      h2biases[i] = random();
      h2neurons[i] = 0;
   }
   for (int i = 0; i<outputDim;i++)
   {
      for (int j = 0; j<h2Dim;j++)
      {
         outweights[j][i] = random();
      }
      outbiases[i] = random();
      output[i] = 0;
   }

}
void forwardProp(float input[inputDim])
{
   StockBear::setZero();
   for (int i = 0; i < inputDim;i++)
   {
      inputLayer[i] = input[i];
   }
   for (int i = 0; i < h1Dim; i++)
   {
      for (int j = 0; j < inputDim; j++)
      {
         h1neurons[i] += input[j] * h1weights[j][i];
      }
```

```
        h1neurons[i] += h1biases[i];
        h1neurons[i] = ReLU(h1neurons[i]);
    }
    for (int i = 0; i < h2Dim; i++)
    {
        for (int j = 0; j < h1Dim; j++)
        {
            h2neurons[i] += h1neurons[j] * h2weights[j][i];
        }
        h2neurons[i] += h2biases[i];
        h2neurons[i] = ReLU(h2neurons[i]);
    }
    for (int i = 0; i < outputDim; i++)
    {
        for (int j = 0; j < h2Dim; j++)
        {
            output[i] += h2neurons[j] * outweights[j][i];
        }
        output[i] += outbiases[i]; //linear output when doing regression
    }

}
void backProp(float target[], float inputs[][inputDim], int batchSize) {
 // Accumulate gradients over the mini-batch
 float outputGradients[outputDim] = {0.0};

 for (int batchIndex = 0; batchIndex < batchSize; ++batchIndex) {
     forwardProp(inputs[batchIndex]);

     // Calculate output layer gradients for each sample in the mini-batch
     for (int i = 0; i < outputDim; ++i) {
         outputGradients[i] += output[i] - target[batchIndex];
     }
 }
error = outputGradients[0]/batchSize;

 // Update output layer weights and biases
 for (int i = 0; i < h2Dim; ++i) {
     for (int j = 0; j < outputDim; ++j) {
         outweights[i][j] -= eta * (outputGradients[j] / batchSize) *
             h2neurons[i];
     }
     outbiases[i] -= eta * outputGradients[i] / batchSize;
 }

 // ... Similar updates for hidden layer 2 and hidden layer 1
```

```
    // Calculate hidden layer 2 gradients
    float h2Gradients[h2Dim] = {0.0};
    for (int i = 0; i < h2Dim; ++i) {
        for (int j = 0; j < outputDim; ++j) {
            h2Gradients[i] += outputGradients[j] * outweights[i][j];
        }
        h2Gradients[i] *= dReLU(h2neurons[i]);
    }

    // ... Similar updates for hidden layer 1

    // Calculate hidden layer 1 gradients
    float h1Gradients[h1Dim] = {0.0};
    for (int i = 0; i < h1Dim; ++i) {
        for (int j = 0; j < h2Dim; ++j) {
            h1Gradients[i] += h2Gradients[j] * h2weights[i][j];
        }
        h1Gradients[i] *= dReLU(h1neurons[i]);
    }

    // ... Similar updates for input layer

    // Update input layer weights and biases
    for (int i = 0; i < inputDim; ++i) {
        for (int j = 0; j < h1Dim; ++j) {
            h1weights[i][j] -= eta * (h1Gradients[j] / batchSize) * inputLayer[i];
        }
        h1biases[i] -= eta * h1Gradients[i] / batchSize;
    }
}
   float predict(float inp[inputDim])
   {
      StockBear::forwardProp(inp);
      return output[0];
   }


   int inputLayer[inputDim];
   float h1weights[inputDim][h1Dim]; //matrix representing weights between input
       and HL1
    float h1biases[h1Dim]; //array representing biases of HL1
    float h2weights[h1Dim][h2Dim]; //matrix representing weights between HL2 and
        HL1
    float h2biases[h2Dim];//array representing biases ofHL1
    float outweights[h2Dim][outputDim];//matrix representing weights between HL2
        and output
    float outbiases[outputDim]; //array representing biases between input and HL1
```

```
 float h1neurons[h1Dim]; //array representing HL1 activations
 float h2neurons[h2Dim]; //array representing HL 2 activations
float trainingExs = 0;
float eta = .005;

void setZero()
{
   for (int i = 0; i < h1Dim; i++)
   {
      h1neurons[i] = 0;
   }
   for (int i = 0; i < h2Dim; i++)
   {
      h2neurons[i] = 0;
   }
   for (int i = 0; i<outputDim;i++)
   {
      output[i] = 0;
   }
}

void saveModelText(string filename) {
 ofstream file(filename);

 for (int i = 0; i < inputDim; ++i) {
     for (int j = 0; j < h1Dim; ++j) {
         file << to_string(h1weights[i][j]) << ",";
     }
     file << "\n";
 }
for (int i = 0; i < h1Dim; ++i) {file << to_string(h1biases[i]) << ",";}
file << "\n\n";
for (int i = 0; i < h1Dim; ++i) {
     for (int j = 0; j < h2Dim; ++j) {
         file << to_string(h2weights[i][j]) << ",";
     }
     file << "\n";
 }
for (int i = 0; i < h2Dim; ++i) {file << to_string(h2biases[i]) << ",";}
file << "\n\n";

 for (int i = 0; i < h2Dim; ++i) {
     for (int j = 0; j < outputDim; ++j) {
         file << to_string(outweights[i][j]);
     }
     file << "\n";
 }
```

```
   for (int i = 0; i < outputDim; ++i) {file << to_string(outbiases[i]) << ",";}
   file << "\n\n";
    file.close();
}
};
int DATAINDEX = 0;
void loadIO(int bs, float inp[][inputDim], float out[])
{
   for (int i = 0;i<bs;i++)
   {
      string fen = DATA[DATAINDEX*2];
      fenToVec(fen, inp[i]);
      out[i] = stof(DATA[DATAINDEX*2 + 1]);
      DATAINDEX++;
   }
}
int main()
{
   StockBear stockbear;
   int EPOCHS = 100;
   const int batchSize = 15;
   float input[batchSize][inputDim];
   float target[batchSize];
   float errorE = 10;
      for (int i = 0; i<EPOCHS; i++)
      {
         loadIO(batchSize, input, target);
         stockbear.backProp(target, input, batchSize);
         cout << "Epoch " << i+1 << "'s error: " << stockbear.error << endl;
      }
   stockbear.saveModelText("stockbear.h");//parsed with python
   return 0;
}
```